MethodMesh Master Book

Current working MethodMesh doctrine

2026-09-06

Version: v1.04
Status: consolidation draft
Last updated: 2026-09-06
Authority: current working MethodMesh doctrine

This book does not introduce new rules. It consolidates the rules MethodMesh is already using so future work has one place to start from.

After this book is reviewed, older overlapping notes can be archived or marked as historical. Until then, do not delete older documents solely because this book exists.

1. MethodMesh in one page

MethodMesh is an offline-first Android toolbox and capability runtime.

It has two jobs:

1. Let a person do useful field tasks directly on a phone.
2. Let external systems such as ODK/XLSForm, presets, protocols, schedules and widgets

call those same tasks through a consistent capability contract.

The product rule is:

Do stuff. Return the beef, keep the salad available when explicitly wanted.

“Beef” means the main useful result:

barcode text; translated text; redacted image; searchable PDF and OCR text; Plus Code; temperature/humidity; navigation result; selected scale value; signed attestation hash.

“Salad” means metadata, provenance, diagnostics, raw JSON, manifests, traces, internal IDs and audit fields. These matter, but they should not dominate the everyday native UI.

2. Current authority order

When documents conflict, use this order until the repository is cleaned up:

1. This Master Book, once reviewed and accepted.
2. Current app behaviour and tests on master .
3. README.md .
4. docs/CAPABILITY_WRITING_GUIDE.md .
5. Current capability README files under each module’s docs/ .
6. Current specifications listed in specifications/SPEC_STATUS.md .
7. Roadmap notes for open issues and future work.

8. Archived specifications and old ResearchOS-era notes.
9. Prototype handoff notes under incoming_capability_prototypes/ .

Archived documents are historical context, not active instructions.

Prototype documents are suggestions until reviewed in Work mode and made to pass the app build and MethodMesh rules.

Release notes describe what happened in a release; they do not define future architecture.

3. Identity and product character

MethodMesh should feel:

practical; minimal; focused; modular; reliable; field-ready.

The visual identity is:

MethodMesh; “Do Stuff”; clean cards; restrained typography;

one accent colour; clear MethodMesh-styled icons; no unnecessary verbosity.

The native app should feel like a toolbox, not a database management platform.

When run natively, a user often wants to:

scan something; translate something; redact and share a photo; capture a location; read a sensor; toss a coin; launch a preset from a widget; get a result and leave.

Do not force database/storage/audit concepts into that path unless the user asks for them.

4. Top-level app model

The app is organised around:

Dashboard; Presets; Protocols; Capabilities; Workbench; Device registry; Settings; Widgets.

Dashboard

Dashboard result interaction

The dashboard is the operational control centre for MethodMesh. It should make useful outputs immediately actionable while still providing fast access to the underlying capability when more control is needed.

Any dashboard value that is meaningfully calculated or returnable should support direct tap-to-copy. The clipboard payload is the primary useful value — the beef — rather than the card label, field name or decorative presentation.

- Plus Code: copy only the Plus Code, not text such as “Plus Code: 8FVC9G8F+6W”.
- Temperature, dew point, humidity, distance and similar scalar results: tapping the displayed result copies the useful value.
- Where a result naturally supports Android sharing, provide a nearby share action without forcing the user through a verbose details screen.

Labels, units, provenance and contextual information may remain visible, but they should not contaminate the primary clipboard return unless the capability explicitly defines them as part of the useful result.

Media, maps and visual outputs

Dashboard components that display an image, map, rendered document, chart or other visual artefact should expose a small, visually restrained action beside the component for sharing and, where appropriate, saving to the device.

Maps and other spatial views must be smoothly pan- and zoom-friendly and should open into a dedicated full-screen view. Returning from full screen should preserve dashboard state.

For map-backed capabilities, the dashboard may provide the compact operational view, current result and common controls while full-screen mode provides the larger manipulation surface. The shared dashboard must consume generic capability-declared result and action contracts rather than learning Plus Code-, navigation- or provider-specific behaviour.

Dashboard as module control centre

To the extent sensible and feasible, a dashboard component should act as the control centre for its module rather than merely a passive summary. It may expose capability-declared quick actions, current state, refresh/run controls, preset entry points, active schedule controls and recent useful results. Deeper configuration should open the capability or appropriate top-level page rather than expanding the dashboard into a long settings form.

The dashboard remains a dashboard, not a long scrolling substitute for real top-level pages. Prefer compact cards, progressive disclosure and direct manipulation. A tired fieldworker should be able to see the state, obtain or copy the useful result, share an artefact, rerun or refresh where appropriate, or open the full capability with minimal navigation.

Dashboard polish requirement

Dashboard interaction quality is a product requirement. Avoid clunky controls, oversized button rows, duplicated labels, modal churn, unnecessary confirmation, raw JSON, dense metadata and inconsistent interaction patterns. Touch targets must remain accessible, but controls should be visually quiet. Transitions into full-screen maps or media and back should preserve state and feel deliberate.

Preferred hierarchy: result first; tap-to-copy or immediate primary action; compact share/save control where applicable; quick operational controls where useful; full capability/details on demand; audit/provenance only when explicitly opened or exported.

The dashboard is for shortcuts and current activity:

MethodMesh header/logo; find a capability; find a preset; common tasks; active schedules with pause/resume toggles; recent useful activity where appropriate.

The dashboard is not a long scrolling substitute for real top-level pages.

Presets

Presets are saved capability setups.

A preset stores configuration choices and runtime-input policy. It should not store accidental user text or run-specific values unless that is genuinely the intended fixed value.

Protocols

Protocols are chains of presets/capabilities.

They need rails. A user should understand:

current step; previous step outcome; next step; whether they need to do anything; how to retry, cancel or continue; final combined result.

Capabilities

One capability contract, multiple interaction surfaces

The dashboard is mandatory, but it is only one interaction surface. It must never become the implementation boundary for a capability.

Every MethodMesh capability is defined once by a canonical module-owned contract. Direct native runs, the dashboard, preset creation, protocol creation, schedules/widgets where applicable, and ODK/XLSForm are adapters or projections over that same contract. They are not separate implementations and they must not drift into different feature sets.

The invariant is: if MethodMesh can perform an operation or produce a declared output, that operation and output remain addressable through the canonical capability contract regardless of which surface invokes it.

Required surface parity

Every module must provide a useful dashboard presence appropriate to the module. The dashboard may aggregate, summarise and expose quick controls, but dashboard aggregation must never hide or replace the individual capabilities.

Every individual capability must remain independently discoverable and selectable when creating presets and protocols. A module with a rich dashboard is not exempt from exposing its underlying primitives for composition.

ODK/XLSForm must be able to invoke every capability and request every declared output that MethodMesh itself can produce, including obscure or rarely used outputs. Example XLSForms may demonstrate only common outputs; the transport contract must not be limited to the examples.

Transport-specific presentation is allowed. For example, a dashboard may render a map, a preset may ask for runtime inputs, and ODK may use namespaced return fields and URI grants. The underlying method ID, input semantics, output field names, types and meanings remain canonical.

Forbidden architectural shortcuts

Do not create dashboard-only capability logic or dashboard-only outputs. Do not create preset/protocol-only functionality that ODK cannot invoke. Do not maintain a second output schema for ODK. Do not reimplement capability behaviour separately in each surface.

Capabilities are production-ready primitives for real use and the canonical units of invocation and composition.

Every individual capability must remain exposed and coherent when run:

directly; as presets; in protocols; in schedules; from widgets; from ODK/XLSForm.

Workbench

Workbench is for development, device, inspector and "hackerish" tools. These may be very useful but are not usually things a field user saves as a normal research preset.

Examples:

ESP32 firmware installation; BLE/Bluetooth inspection;

Android app inspection; API definition editing and testing; hardware diagnostics; prototype/development capabilities.

Device registry

The device registry should eventually be a live useful view, not only a static list.

For sensors it should support refreshing and showing current key values, such as:

AHT20: temperature and humidity; LD2410C/LR radar: presence, target state, distances and energy values.

Settings

Settings is for shared services and app-level controls, not capability-specific special cases.

Examples:

downloaded ML Kit language packs; output/export preferences; future offline map tile packs; shared device/service controls.

Widgets

Android desktop widgets should support:

1×1 preset trigger; 1×1 protocol trigger; 1×1 schedule toggle.

Preset/protocol widgets run the thing. Schedule widgets toggle on/off.

Widget-launched one-shot actions should return to the Android desktop after completion rather than taking the user back into app navigation.

5. The golden rule

The shared MethodMesh UI knows nothing about individual capabilities.

This is the most important engineering rule.

The core app may know about:

capability discovery; generic settings rendering; preset creation; runtime inputs; result actions; ODK/external transport; protocols; schedules; widgets; shared services; broad categories and icon keys.

The core app must not learn bespoke facts such as:

barcode scanners have format choices; image redaction has grid rows; printers need paper width; translation has source and target language; Plus Code capture has satellite mode; sensors have AHT20 or LD2410C fields.

Those belong inside the capability module.

If the shared UI needs to change, ask:

Is this a generic framework improvement that helps all capabilities?

If yes, it belongs in shared code. If no, keep it in the module.

6. Capability module contract

Every capability lives under:

app/src/main/java/com/example/methodmesh/modules/<module_name>/

The canonical handoff unit is one folder.

Returned module designs and implementation handoffs must contain the module folder itself, not a reconstructed tree for the whole MethodMesh app or repository. The receiving chat or Work-mode review should be able to drop, inspect or adapt that one folder without separating it from synthetic app scaffolding.

Do not reproduce app/, project-level Gradle files, HomeScreen, shared navigation, repository root files or other unrelated trees merely to show where the module would live. If a genuine shared-framework change is required, describe it explicitly as a separate integration requirement; do not smuggle it into the returned module tree.

Canonical returned folder structure

The returned artefact must have **exactly one top-level folder**, named for the module:

```
<module_name>/
|-- docs/
|   |-- README_<CapabilityA>.md
|   |-- example_odk_<capability_a>.xlsx
|   |-- README_<CapabilityB>.md              # if the module exposes another capability
|   |-- example_odk_<capability_b>.xlsx      # one example per capability
|   |-- VALIDATION.md                        # recommended where validation is meaningful
|   |-- ROADMAP_NOTE.md                      # optional
|   |-- THIRD_PARTY_NOTICES.md               # when required
|   `-- ATTRIBUTION.md                       # when required
```

```
|-- <ModuleName>Module.kt                    # required: MethodMeshModule entry point
|-- <CapabilityA>Method.kt                   # required: one canonical method implementation
|-- <CapabilityA>CapabilityScreen.kt         # required when the capability has native interaction
|-- <CapabilityB>Method.kt                   # if the module exposes another capability
|-- <CapabilityB>CapabilityScreen.kt         # if that capability needs its own native screen
|-- <Capability>Repository.kt                # only when state/data access justifies it
|-- <ModuleName>DashboardCard.kt             # only if generic dashboard rendering is insufficient
`-- <other module-owned helpers>.kt          # only when genuinely module-owned
```

The folder shown above is the **handoff root**. When admitted to the repository, that folder belongs at:

`app/src/main/java/com/example/methodmesh/modules/<module_name>/`

Do **not** include `app/`, `src/`, `main/`, `java/`, `com/`, `example/`, `methodmesh/` or `modules/` as wrapper directories in the returned artefact. The receiver already knows the destination.

If the handoff is zipped, the ZIP must open to exactly one top-level `<module_name>/` directory. Do not add an extra wrapper such as `MethodMesh/`, `<module_name>_handoff/`, `app/` or `modules/`.

File placement rules

- `<ModuleName>Module.kt` is the single module entry point and owns discovery-facing module metadata.
- Each independently invokable capability must have a clearly identifiable canonical method implementation inside the module.
- Native screens, dashboard components, repositories and helpers stay beside the module code. They are implementation details of that module, not shared-app files.
- `docs/` contains **all documentation and ODK examples for the module**. Do not return a second documentation tree elsewhere.
- Because every MethodMesh capability must be accessible through the ODK/XLSForm roundtrip, provide an ODK example for every individual capability. A combined example workbook is acceptable only if it clearly demonstrates every method; otherwise use one `example_odk_<capability>.xlsx` per capability.
- For a multi-capability module, repeat the per-capability README, ODK example, method implementation and native screen as appropriate. Do not collapse independent capabilities into a single private dashboard implementation.
- A custom dashboard file is optional. The **dashboard presence is not optional**. If generic shared rendering can provide that presence from module metadata, no custom dashboard file is needed.
- `Repository.kt` is optional. Do not create repository/service/helper layers merely to make the folder look architecturally elaborate.
- Module-specific helpers belong in this folder. Shared framework changes do not.

Non-code artefacts and exceptional integration

If a capability genuinely requires a bundled non-code artefact such as firmware, a model, a template or static data, include it inside a clearly named module-local subfolder **only if the current MethodMesh architecture already supports loading that artefact from the module**.

If current architecture requires a file to live elsewhere in the repository, do not fabricate that repository tree in the handoff. Describe the required placement as an explicit integration requirement outside the returned folder.

The same rule applies to genuinely necessary shared-framework changes: state the change separately in the handoff notes. Do not return modified `HomeScreen`, shared navigation, project Gradle files or other central files as though they were part of the module.

Handoff cleanliness

Do not hand back:

- loose Kotlin files;
- a separate top-level `docs/` folder;
- a reconstructed whole-app or repository tree;
- duplicated wrapper folders;

- unrelated app/shared/framework files included merely for context;
- APKs, build outputs, Gradle caches or generated intermediates;
- IDE/editor metadata such as `.idea/`;
- operating-system metadata such as `.DS_Store` or `__MACOSX/`;
- a ZIP whose extraction location or intended module root requires guessing;
- a module that needs central UI special-casing;
- a module with an individual capability that cannot be selected for presets/protocols or invoked through ODK.

Auto-discovery

Modules expose one object implementing MethodMeshModule .

The app discovers modules automatically. Do not add a one-off central registration list.

Module-owned metadata

A module owns:

method IDs; descriptors; settings; output fields; capability screens; docs; examples; dependencies; icon hint; tests where possible.

Canonical capability contract

A capability declares its method ID, inputs, settings, runtime-input policy, outputs, result types, closeout semantics and applicable actions once in module-owned code/metadata. Shared surfaces consume that declaration.

The declaration is the source of truth for dashboard actions, direct native execution, preset authoring/execution, protocol composition, schedules/widgets where applicable, and ODK/XLSForm transport.

A surface may choose how to present or transport a value, but it must not silently remove capability functionality. If a declared output exists, generic result projection must make it addressable to ODK and to other MethodMesh composition surfaces even when it is not normally shown in the everyday UI.

Output field names, types and semantics are canonical. ODK namespaces, media attachment handling, clipboard formatting and dashboard rendering are projections of those canonical fields, not new field definitions.

Icon key

Modules may expose an iconKey for generic surfaces such as widgets.

Good broad keys:

document ; location ; language ; hardware ; random ; schedule ; tool .

This is still generic. It does not teach the dashboard about a specific capability.

7. Capability lanes

Capabilities move through lanes:

Incoming prototype

Prototype code from another chat or contributor that has not been reviewed.

Location:

incoming_capability_prototypes/

Rules:

do not auto-discover it; do not assume it builds; do not trust its architecture; treat docs as suggestions.

Development

Builds and is admitted into modules/ , but not yet polished for production.

Development capabilities may be visible in Workbench/development areas.

They must not claim production status unless they pass the production checklist.

Production

A production capability:

builds; runs; has native UX checked; has preset UX checked; has ODK/XLSForm checked; preserves state across rotation where relevant; returns beef-first outputs; has docs and example XLSForm; does not violate the golden rule; has any required attribution/licence notes; has a clear production method status.

Workbench

Workbench contains tools that are useful, technical, diagnostic or operational, but not normal field primitives.

Examples:

firmware installer; sensor provisioner; app inspector; Bluetooth inspector; API definition editor; debug tools.

Workbench items can be stable without being "production capabilities" in the end-user sense.

8. Settings and runtime inputs

Capability settings must be declared through MethodSetting .

Use:

BooleanSetting for toggles; ChoiceSetting for dropdown/radio-style choices; MultiChoiceSetting for checkbox groups; IntSetting and FloatSetting for numbers; TextSetting only for genuine free text.

Do not use raw text boxes for:

barcode formats;

language choices; output modes; yes/no settings; fixed enumerations; source selectors; known device/service modes.

Fixed, runtime and operational inputs

Each setting should conceptually be one of:

1. Fixed preset configuration.
2. Runtime input.
3. Operational control.

Fixed preset configuration is hidden when the preset runs natively.

Runtime input is asked before the action runs.

Operational controls may remain visible when genuinely necessary, such as changing or reconnecting a Bluetooth device.

Preset authoring

Preferred flow:

configure/test → pretty config screen → test
configure/test → same pretty config screen → save as preset → choose fixed/runtime settings → name preset

The preset screen should not be a second inferior text-box version of the test screen.

9. Native run UX

Native MethodMesh is a toolbox.

Native runs should:

ask only what is needed; require a clear start/submit/confirm before meaningful work; run automatically where the preset already supplies all required settings; preserve result state across orientation changes; show a clear result screen;

put the main result at the top; hide original input/settings/details behind expandable sections where useful; provide actions: share, copy, save to Downloads, done/home, retry where relevant; avoid automatic internal saves; keep JSON behind an explicit "include full JSON" style option.

Native runs should not:

dump huge JSON on the user by default; jump back to home without clear completion; auto-save files to internal folders as the default; show fixed preset settings again; make buttons act like checkboxes; use long verbose explanatory text where labels suffice.

Sharing rule

Share only the beef by default.

Examples:

barcode scan shares only the decoded payload; Plus Code capture shares only the Plus Code; translation shares only the translated text; image redaction shares only the redacted image; document scanner shares the chosen document/text output; sensor read shares "temperature 24, humidity 55%" style values; conversation translator shares the transcript.

If the user toggles full JSON/audit inclusion:

share includes beef + media + JSON file; save to Downloads includes beef/text + media + JSON file; copy includes beef text + JSON text.

With the toggle off:

share/save/copy include only beef and media.

Done/Home rule

The old "Submit" label is not right for ordinary native completion. Use "Done" unless the action is genuinely submitting to another system.

Native Done should finish the flow.

Widget-launched Done should return to the Android desktop.

App-launched Done/Home should return to the MethodMesh dashboard, not a preset launch page.

10. ODK/XLSForm contract

ODK calls MethodMesh through Android intents, normally from an XLSForm group.

Rules:

intent calls should be made via groups; inputs use input_* intent parameters; return fields are group children; blank return fields must not overwrite request parameters; ODK supplies its own UI and values; MethodMesh should not force ODK callers through native setup dialogs; MethodMesh should store nothing merely because it is handling an ODK contract; MethodMesh returns data to ODK and lets ODK own the submission.

ODK capability parity

ODK is a first-class invocation surface for the same MethodMesh capability contract. It is not a reduced API containing only the outputs currently used by example forms.

Anything MethodMesh can do through a declared capability must be invokable from ODK where Android/ODK transport can represent the interaction. Anything a declared capability can return must be requestable/returnable to ODK, however obscure the field may be and regardless of whether a real-world form is expected to use it.

This requirement applies to scalar values, structured values, hashes, diagnostics/audit fields, media/file URIs, calculated values and other declared outputs. Where a type requires transport machinery, such as ClipData and read grants for binary artefacts, the transport layer provides that machinery without changing capability semantics.

Example XLSForms are examples, not allow-lists. Adding a new capability output does not require a bespoke ODK code path; the generic projection/transport layer must expose it from the canonical output contract.

Namespaces

ODK forms may use:

methodmesh_return_namespace='photo'

The namespace projector prefixes returned keys:

redacted_image_uri → photo_redacted_image_uri redacted_image_sha256 → photo_redacted_image_sha256 methodmesh_full_json → photo_methodmesh_full_json

Do not redesign namespace handling casually. It exists to avoid field collisions when several MethodMesh calls appear in one form.

Flat + full JSON

The ODK default pattern is:

main useful field(s) + methodmesh_full_json

For media-producing capabilities:

main media URI + useful scalar fields + methodmesh_full_json

Example:

redacted_image_uri redacted_image_sha256 methodmesh_full_json

Binary artefacts

For returned binary artefact content:// URIs:

keep the namespaced string extra; add the URI to returned Intent ClipData ; set Intent.FLAG_GRANT_READ_URI_PERMISS
; do this centrally in transport, not in the capability.

This lets ODK import files as real attachments rather than only receiving URI text.

No MethodMesh storage on ODK return

When handling ODK/external-app contracts, MethodMesh should not create extra output- folder saves. It should return the result/URI/grant and let the caller own persistence.

Temporary/cache files needed to expose an artefact through FileProvider are acceptable as artefact source files. Extra "just in case" MethodMesh archive copies are not.

11. Output contract

Every capability should define:

core result fields; optional audit/ALCOA fields; full JSON representation; media/file fields where relevant; error fields.

Cross-surface output invariant

Every declared output belongs to the capability contract, not to a UI surface. A field may be hidden from the ordinary native result screen, omitted from a compact dashboard card, or uncommon in presets, but it remains addressable for protocols, pipes and ODK projection.

Beef-first governs presentation and defaults; it does not authorise deleting salad or obscure outputs from the contract. Presentation can be selective. Capability availability cannot.

Core result

The core result is what most users care about.

Examples:

barcode_payload ; plus_code ; mlkit_translate_text ; redacted_image_uri ; redacted_image_sha256 ; document_scan_searchable_pdf_uri ; document_scan_ocr_text ; lower_value , upper_value ; conversation_transcript .

Audit/ALCOA fields

Audit fields support:

attributable; legible; contemporaneous; original; accurate.

Common audit fields include:

time; method ID; execution ID; device/source information; configuration values; selected cell/region definitions; hashes; manifests; status; diagnostics.

Full JSON

Full JSON is the complete auditable payload.

It should be available:

as methodmesh_full_json for ODK where requested; as opt-in export/share metadata for native runs; as background data for verification workflows.

It should not be the primary native result screen.

Hashes

Hashes should represent the actual object being committed.

Do:

hash final image bytes for image artefacts; hash exact UTF-8 bytes for text commitments; hash exact recipe strings where the recipe itself is part of the commitment; stream large files where possible.

Do not:

hash a URI string when the claim is about file bytes; hash pretty-printed JSON if the commitment is to exact supplied JSON; silently normalize, trim or rewrite commitment text unless the contract says so.

12. Capability closeout, protocols and schedules

Capabilities need a closeout contract.

A capability step should end with one of:

completed with payload; completed without payload; failed with diagnostic; cancelled; externally completed and manually confirmed; retry requested.

Single native preset

A single manually run preset should usually run without protocol rails.

It should show the result and actions:

share; copy; save to Downloads; Done; retry if useful.

Protocol step

Protocols should guide the user:

Previous step completed: barcode scanned Next step: random number Go

For external apps with no callback:

Next step: complete web form Go … Did you complete the web form? Yes / No / Cancel

If Yes, continue.

If No, offer retry.

If Cancel, confirm before killing the whole chain.

Final protocol result

The final protocol result should collate the beef from all steps:

barcode: DKJDALSD999 random number: 0.233214 photo: redacted_image.jpg

Metadata should be available behind details/export, not tangled into the main result list.

Schedules

Schedules using presets must use the same closeout contract. They should not get stuck on a preset result screen unless the schedule explicitly requires manual confirmation.

Schedule widgets toggle schedules on/off.

13. Presets, protocols and pipes

Presets are reusable single actions.

Protocols are ordered chains.

Pipes let outputs from one step become inputs to another.

Examples:

AHT20 current temperature → API call outside feels-like temperature → calculation → return all three values

This requires result references that can address:

scalar values; media; structured JSON subtrees; previous step outputs; cached API data.

The online data architecture's ResultTree and path semantics are the right direction. Do not flatten complex provider responses too early.

14. Online data and APIs

api.get is the runtime capability for declared API definitions.

Individual API providers should usually be data definitions, not bespoke capabilities.

The API definition editor/tester belongs in Workbench.

Bundled APIs

Bundled APIs are useful where mature and stable.

They should:

be declarative; expose sensible inputs; show available result paths; support selected/multi-field returns; show source/update age; cache responses; refresh from network by default; fall back to cache if offline or failed where policy allows.

Location privacy

Where API calls use location, default to rounded disclosure when appropriate.

Current accepted rule:

5 km rounded coordinates for declared third-party API providers

Do not silently send exact GPS to remote services.

Credentials

Credential rules:

reference credentials, do not embed them in exported definitions; redact credentials in display URLs/logs; do not include secrets in ODK payloads, provenance or debug logs.

RSS/Atom

RSS and Atom should share API/cache infrastructure but need a specialised reader surface:

subscriptions; read/unread/saved; item identity; deduplication; offline search; media policy.

Video auto-download is off by default.

15. Offline-first rule

MethodMesh should work offline where practically possible.

Offline-first does not mean online is forbidden. It means the core useful operation should not depend unnecessarily on a proprietary or remote service.

Examples:

Plus Codes are calculated locally. Conversation translation uses downloaded ML Kit language packs. Image redaction is local. Document scanning is local where ML Kit/device services allow. GPS navigation works from coordinates or Plus Codes.

Sensor read/provisioning is local BLE/device work.

Online services may improve context:

map tiles; satellite imagery; weather/hazard APIs; TSA timestamping; language pack downloads.

But if online fails, the app should fail clearly or fall back to cached/local behaviour where possible.

16. Shared device services

Settings owns shared device services.

ML Kit language packs

Language packs are shared between language capabilities:

translation; conversation translation; future language/NLP tools.

Settings should show:

Downloaded English (En) [size] trash French (Fr) [size] trash

Tap to download Afrikaans (Af) down arrow Albanian (Sq) down arrow …

Downloaded languages appear at the top.

Language names should be human-readable first, code second:

English (En) French (Fr) Swahili (Sw)

Use canonical ML Kit-supported language codes. Some language labels, such as Chinese variants, may need careful canonical mapping rather than assuming a colloquial label is a valid pack ID.

Download UI should show activity and debug/progress state clearly enough to tell the difference between "network off" and "stuck".

Respect Google attribution and terms for ML Kit features.

Future map tiles

Offline map tile management should eventually follow a similar shared-service pattern:

download; delete; region; size; attribution; online fallback; offline availability.

17. Location tools

Plus Code capture

Principle:

GPS → local/offline-capable map view → OLC grid → user selects cell → store full Plus Code

Rules:

Open Location Code calculation must be local and deterministic. Do not depend on Google APIs or what3words. Store full globally self-contained Plus Codes. Basemaps are contextual only. Online maps may be default while offline maps are not yet implemented. Street/satellite/grid modes can help selection. Satellite imagery is useful for building selection. Selected cell should map correctly to real-world OLC geometry, not merely screen projection. ODK return should be Plus Code plus metadata JSON.

Native share should be only the Plus Code.

GPS target navigator

Rules:

target may be lat/lon or Plus Code; preset mode should ask for lat/lon or Plus Code then start navigation; navigator points to target, not north; distance must show current distance; AR camera should be a separate view with crosshair/HUD; orientation should work upright as required for AR, not only flat.

18. Sensors and hardware

Hardware/device tools belong mostly in Workbench unless they are polished field-facing capabilities.

ESP32 sensor work includes:

firmware installer; sensor provisioner; sensor read; device registry; live diagnostics.

Rules:

sensor firmware and app protocol must agree on installed profiles; provisioning must not retain stale names or stale profiles after proper reset; live sensor read should show key values rather than huge manifests; device registry should have refresh/live view; firmware images that are core to the system should be tracked; manual docs videos need not be tracked if updated outside the repo.

19. Attestation and audit commitments

The current attestation direction is:

ODK constructs canonical commitment string ODK hashes it ODK sends event_payload_hash + commitment_recipe to MethodMesh MethodMesh signs/timestamps those commitments Verifier reconstructs later

MethodMesh should not require ODK to send every raw field value into attestation.create .

attestation.create

Inputs:

event_payload_hash ; commitment_recipe ; verification method/evidence; timestamp policy; optional study/operator/subject/event metadata.

Rules:

ODK owns payload construction. MethodMesh validates event_payload_hash as SHA-256 hex. MethodMesh validates commitment recipe JSON. MethodMesh calculates commitment_recipe_sha256 . The signed canonical attestation binds event_payload_hash and commitment_recipe_sha256 . Large objects do not pass through attestation. Media artefacts are represented by byte hashes, such as redacted_image_sha256 .

Commitment recipe

Recipe schema:

methodmesh.commitment_recipe.v1

Canonicalization:

ordered-kv-v1

Hash:

SHA-256

Supported commitment types:

value ; artifact-bytes-sha256 ; text-utf8-sha256 ; json-utf8-sha256 .

The recipe is declarative. Do not execute expressions, XPath, JavaScript, ODK calculations or arbitrary code inside it.

Trusted timestamp

RFC 3161 trusted timestamping may be:

disabled; preferred; required.

If required and unavailable, the attestation should fail clearly rather than silently creating an incomplete trusted timestamp claim.

20. Documentation rules

Every admitted capability should have module-local docs. All of those documents live inside the returned module's `docs/` directory; there is no parallel top-level documentation handoff.

For a module exposing several capabilities, documentation is capability-addressable: each method must be identifiable in the docs and each capability must have an ODK example that exercises its canonical contract.

Minimum:

- `docs/README_<Capability>.md`
- `docs/example_odk_<capability>.xlsx`

Recommended:

- `docs/VALIDATION.md`
- `docs/ROADMAP_NOTE.md`
- `docs/THIRD_PARTY_NOTICES.md`
- `docs/ATTRIBUTION.md`

Capability README should cover:

purpose; method ID;

lane/status; native UX; preset behaviour; ODK/XLSForm behaviour; inputs/settings; runtime inputs; outputs; core result fields; audit fields; full JSON field; media/attachment rules; permissions; offline/online behaviour; dependencies; examples; validation notes; attribution/licensing.

Capability documentation must explicitly state that the dashboard is one projection of the capability contract and must identify how each individual capability is exposed for direct native use, presets, protocols and ODK/XLSForm.

README output tables should distinguish "normally shown" from "contractually available" where helpful, but all declared outputs must remain part of the canonical contract.

Example XLSForms

Example XLSForms should:

use grouped intent calls; avoid return/input field name collisions; include main output fields; include method-mesh_full_json where audit metadata is expected; include media fields as real ODK attachment types where relevant; demonstrate realistic usage, not just a synthetic debug call.

Do not rewrite all example forms during architecture experiments unless explicitly asked. First prove the transport/contract works.

21. Testing rules

Run tests proportional to risk.

For focused changes:

add focused tests at the contract boundary; run those tests; run :app:assembleDebug .

Examples of contract-boundary tests:

every registered capability is exposed to preset discovery; every registered capability is exposed to protocol discovery; every registered capability is representable to ODK transport; every declared output can be projected by ODK; output names/types/semantics are unchanged across projections; dashboard aggregation does not suppress underlying capability discovery; namespace projection preserves keys; binary artefact returns include ClipData/read grant; hash is computed over final bytes; recipe validation rejects malformed input; preset fixed settings hide at runtime; orientation state survives.

For capability promotion:

Contract-parity tests should be generic wherever possible: enumerate registered methods and declared outputs, then assert that dashboard discovery, preset discovery, protocol discovery and ODK projection are derived from the same canonical metadata rather than separate hand-maintained lists.

build debug APK; exercise the dashboard presence; exercise direct native run; verify every individual capability appears in preset creation; verify every individual capability appears in protocol creation; exercise native preset run; verify ODK can invoke each method and project every declared output; exercise the example XLSForm where possible; check canonical field names/types/semantics match across surfaces; check share/copy/save behaviour; check no golden-rule or contract-parity violation.

22. Build, release and git hygiene

Common build:

./gradlew :app:assembleDebug

Common test/build:

./gradlew :app:testDebugUnitTest ./gradlew :app:assembleDebug

Release process normally includes:

update docs/release notes; run relevant tests; build debug; commit; push; tag; GitHub release.

Dirty worktree rule

The MethodMesh worktree is often busy.

Before committing:

inspect status; stage only the coherent intended set; do not accidentally stage prototypes, .idea , firmware backups or unrelated docs; preserve user/other-chat changes unless explicitly asked to clean them up.

Destructive cleanup

Do not delete, reset or archive broad sets of files without explicit confirmation.

Archiving old docs should be done as a deliberate pass after this Master Book is reviewed.

23. Third-party and attribution rules

Capabilities using third-party libraries, data or services must document:

provider; licence/terms; attribution requirements; offline/online dependency; data sent off-device; credentials/API keys; privacy implications.

Important known areas:

Google ML Kit translation/vision/document scanning; Open Location Code / Plus Codes; OpenFreeMap; Esri World Imagery; OpenStreetMap-derived tiles; Open-Meteo; GDACS; World Bank Indicators; exchange-rate providers; RFC 3161 timestamp authorities.

OpenStreetMap volunteer tile servers should not be used in a way that violates tile usage policy. Prefer appropriate tile providers or offline packs.

24. Current production capabilities

Production status should be derived from current method metadata and recent review. The known polished production set includes:

barcode.scan — barcode/QR scanning; calibrated_scale — calibrated on-screen scale; document.scan — document scanning/OCR/PDF; gps_target_navigator — target navigation and AR guide; plus_code.capture — Plus Code

capture; image.redact — image redaction; admin_fingerprint_confirmation — local device authentication; conversation.translate — live conversation translator; mlkit.translate — text translation; random.number.generate — random number generation; mlkit.vision.analyze — ML Kit vision; odk_form_launcher — ODK form launcher.

Some capability names/method IDs have historical aliases or mismatches. Use the method ID in code and docs.

25. Known consolidation problems

These are not new tasks; they are known areas where the repo still needs cleanup.

ResearchOS hangovers

Some archived specifications and old language still describe ResearchOS-like systems or older MethodMesh concepts. These should be archived or clearly labelled historical after this book is accepted.

Too many scattered docs

There are many README, DROP_IN, PATCH, VALIDATION, ROADMAP and release-note files. Some are useful; some are prototype sediment.

Needed cleanup:

keep module README and validation docs; archive old patch notes once folded into README/changelog; keep release notes as release history;

mark prototype notes as non-authoritative; keep current specifications, but link them beneath this book.

Capability lanes may not match UI lanes

Some capabilities have metadata saying Development/Production but may not appear in the matching UI lane. Lane display must derive from current metadata consistently.

Exchange-rate API bug

Known issue: exchange-rate currency selection has duplicate/ambiguous currency labels and broken value handling with non-1 input amounts. This remains on the roadmap.

Home/Done flow

Known issue: preset Home/Done routing has had regressions. The rule is clear: app- launched Done returns to dashboard; widget-launched Done returns to desktop.

Closeout contracts

The closeout contract has improved but remains a central architectural priority. Protocols and schedules need the same clean completion semantics as individual presets.

Docs archive pass

Do not archive yet automatically. Proposed future process:

1. Review this Master Book.
2. Decide authority wording.
3. Move historical docs into an archive folder.
4. Add a short index explaining what remains active.
5. Update README to point to this book.

26. Contributor/AI-chat quick rules

If another AI chat is writing a capability, give it these rules:

1. Work from the GitHub repo and this Master Book.
2. Return exactly one module folder - not a whole-app/repository tree, not app scaffolding, and not unrelated shared files. State any genuinely required framework integration separately.
3. Make that returned folder itself `<module_name>/`. If zipped, the ZIP must contain exactly that one top-level folder - no `app/`, `modules/`, repository wrapper, or second handoff wrapper.
4. Follow the canonical return structure in Section 6: module entry point and capability code at the folder root; all documentation and ODK examples under `docs/`; optional repositories/helpers only when justified; no build/IDE/OS junk.
5. For a multi-capability module, include clearly identifiable per-capability method implementations and documentation/ODK coverage. Dashboard aggregation must not collapse those capabilities into a single inaccessible private implementation.
6. Put docs and ODK example XLSForm(s) inside that folder.
7. Do not edit HomeScreen for capability-specific logic.
8. Do not add central capability registration.
9. Declare settings with MethodSetting .
10. Use toggles/dropdowns/checkboxes for fixed choices.
11. Implement one canonical capability contract and project it into every required surface; do not implement separate dashboard/native/preset/protocol/ODK versions.
12. Always provide a dashboard presence, but never make the dashboard the only way to invoke a capability.
13. Expose every individual capability independently for preset creation and protocol creation, even when the dashboard aggregates several capabilities.
14. Make every declared capability and every declared output available to the ODK/XLSForm roundtrip; examples are not allow-lists.
15. Keep method IDs and input/output field names, types and semantics canonical across all surfaces; only presentation/transport may differ.
16. Return beef first, JSON/audit second; presentation selectivity must not remove obscure outputs from the contract.
17. Preserve state across rotation where relevant and do not auto-save internal files by default.
18. Include attribution/permissions/offline notes.
19. Start as Development unless explicitly promoted.
20. If it cannot build the app, put the result in incoming_capability_prototypes/ .
21. Expect Work-mode review before admission.
22. Before handoff, verify dashboard + direct capability + preset + protocol + ODK are all projections of the same contract and that none has a private feature or output schema.

27. The MethodMesh taste test

Before merging any change, ask:

Is there a dashboard presence without making the dashboard the implementation boundary? Can every individual capability still be selected for a preset? Can every individual capability still be selected as a protocol step? Can ODK invoke the same method and request every declared output? Are all of those surfaces using one canonical contract rather than parallel schemas? Is the handoff itself exactly one self-contained `<module_name>/` folder with code at

its root and all module docs/ODK examples beneath `docs/`? If zipped, does it extract to exactly that one top-level folder? Are exceptional shared-framework changes called out separately rather than embedded in a fake whole-app tree?

Does this make the phone more useful in the field? Does it keep the UI simple? Does it return the beef first? Does it keep audit data available without making it obnoxious? Does it work offline where it reasonably should? Does it respect ODK's constraints? Does it avoid storing things MethodMesh does not need to store? Does it preserve the golden rule? Does it compose with presets, protocols, schedules and widgets? Would a tired fieldworker understand what to press next?

If yes, it probably belongs.

If no, it probably needs more MethodMesh-ing.